Taller de recuperación

Navegación en Expo Router: Stack, Slot, Tabs y Drawer

> **Modalidad:** individual.
>
> **Se permite:** la documentación oficial de Expo Router (docs.expo.dev/router) y su propio código del semestre.
>
> **Entrega:** este documento con las Partes A y C respondidas, las 4 capturas de la Parte B pegadas, y el proyecto (enlace al repositorio).

Distribución del tiempo y puntaje

Parte	Contenido	Tiempo	Puntos
0	Lectura de los conceptos clave	5 min	—
A	Teoría: selección múltiple y preguntas abiertas	15 min	15
B	Práctica: app con los cuatro navegadores	30 min	25
C	Depuración: encontrar y corregir errores	10 min	10
	Total	**60 min**	**50**

Nota final = puntos ÷ 10 (escala de 0,0 a 5,0).

Parte 0 · Conceptos clave (5 min)

Lean esta sección antes de empezar; la pueden consultar durante todo el taller.

1. Los archivos son la navegación

En Expo Router cada archivo dentro de `app/` es una ruta. La carpeta define la URL y el archivo `_layout.tsx` define **cómo** se navega entre sus hermanos.

En proyectos nuevos esa carpeta es `src/app/`; en proyectos anteriores es `app/` en la raíz. Las reglas son las mismas; en este taller escribimos `app/` para abreviar.

Archivo	URL	Nota
`app/index.tsx`	`/`	Ruta por defecto de la carpeta
`app/perfil.tsx`	`/perfil`	
`app/libro/[id].tsx`	`/libro/7`	Ruta dinámica: `id = "7"`
`app/(tabs)/favoritos.tsx`	`/favoritos`	Los grupos `( )` no aparecen en la URL
`app/_layout.tsx`	—	No es una pantalla: define el navegador

2. Los cuatro layouts

Layout	Import	Qué hace	Cuándo usarlo

Slot	`expo-router`	Renderiza la ruta hija activa. Sin header, sin historial y sin animación.	Envolver rutas con un diseño o contexto común sin agregar navegación.
Stack	`expo-router`	Pila: `push` pone una pantalla encima, `back` la quita. Header y animación nativos.	Flujos con profundidad: lista → detalle, pasos de un formulario, modales.
Tabs	`expo-router`	Barra inferior con destinos hermanos. Cada pestaña conserva su estado.	3 a 5 secciones principales de uso frecuente.
Drawer	`expo-router/drawer`	Menú lateral deslizable. Requiere `@react-navigation/drawer`.	Muchos destinos o secciones secundarias (ajustes, ayuda, admin).

```
import { Slot, Stack, Tabs } from 'expo-router';
import { Drawer } from 'expo-router/drawer';

<Slot />                                        // sin .Screen, sin opciones
<Stack><Stack.Screen name="perfil" options={{ title: 'Perfil' }} /></Stack>
<Tabs><Tabs.Screen name="index" options={{ title: 'Inicio' }} /></Tabs>
<Drawer><Drawer.Screen name="ajustes" options={{ drawerLabel: 'Ajustes' }} /></Drawer>
```

3. Moverse entre pantallas

API	Efecto
`<Link href="/perfil">`	Enlace declarativo. Con `asChild` envuelve un `Pressable`.
`router.push('/libro/3')`	Agrega una pantalla a la pila; se puede volver.
`router.replace('/')`	Reemplaza la pantalla actual; **no** se puede volver a ella.
`router.back()`	Regresa a la pantalla anterior (o cierra un modal).
`useLocalSearchParams()`	Lee los parámetros de la ruta, por ejemplo `{ id }` en `[id].tsx`.

4. Anidar navegadores

Cada `_layout.tsx` crea un navegador, y ese navegador puede ser **una pantalla** del navegador de arriba. Este es el árbol que van a construir en la Parte B:

```
Stack            app/_layout.tsx
├── (app)        Drawer   app/(app)/_layout.tsx
│   ├── (tabs)   Tabs     app/(app)/(tabs)/_layout.tsx
│   │   ├── index         Inicio (lista de libros)
│   │   ├── favoritos
│   │   └── perfil
│   └── ajustes
├── (auth)       Slot     app/(auth)/_layout.tsx
│   └── login
├── libro/[id]            Detalle (encima de todo)
└── acerca                Modal
```

- Una pantalla se dibuja **dentro** del navegador al que pertenece. Como `libro/[id]` es hija del Stack raíz, al abrirse tapa el Drawer y las pestañas. Si estuviera en `(tabs)/`, la barra de pestañas seguiría visible.
- Cuando un navegador está dentro de otro, los dos dibujan su header. Para no tener dos headers se usa `headerShown: false` en uno de los dos.

Parte A · Teoría (15 puntos)

A1. Selección múltiple (1 punto cada una)

Escriban la letra de cada respuesta en la tabla del final de la sección.

1. ¿Qué hace `<Slot />` en un `_layout.tsx`?

a) Crea una barra de pestañas con las rutas hijas.

b) Renderiza la ruta hija activa, sin header, sin historial y sin animación de transición.

c) Apila las pantallas hijas y muestra un botón para volver.

d) Reserva un espacio vacío para cargar una pantalla más adelante.

2. ¿Cuál es la URL del archivo `app/(tabs)/perfil.tsx`?

a) `/(tabs)/perfil`

b) `/tabs/perfil`

c) /perfil

d) `/(tabs)`

3. ¿Desde dónde se importa el componente `Drawer`?

a) `expo-router`

b) `react-native`

c) `@react-navigation/native`

d) expo-router/drawer

4. Después de un inicio de sesión exitoso, ¿qué instrucción evita que el usuario vuelva al login con el botón atrás?

a) router.replace('/')

b) `router.push('/')`

c) `router.back()`

d) `<Link href="/">`

5. Su app tiene `app/(tabs)/_layout.tsx` con `Tabs` dentro del Stack raíz. Quieren que el detalle de un producto se abra **sin** la barra de pestañas. ¿Dónde crean el archivo?

a) `app/(tabs)/producto/[id].tsx`

b) app/producto/[id].tsx, como pantalla del Stack raíz

c) Dentro de `app/(tabs)/_layout.tsx`, como un `Tabs.Screen` más

d) No se puede: la barra de pestañas siempre es visible

6. En `app/libro/[id].tsx`, al navegar a `/libro/42`, ¿cómo se lee el parámetro?

a) `props.route.params.id`

b) `useRouter().id`

c) const { id } = useLocalSearchParams()

d) `useSegments()[0]`

7. El archivo `app/(tabs)/editar.tsx` debe seguir siendo una ruta pero **no** debe aparecer en la barra de pestañas. ¿Qué opción usan en su `Tabs.Screen`?

a) `tabBarIcon: null`

b) `headerShown: false`

c) `presentation: 'modal'`

d) href: null

8. Un `Tabs` dentro de un `Stack` muestra dos headers, uno encima del otro. ¿Cuál es la corrección más adecuada sin perder la navegación de pila?

a) options={{ headerShown: false }} en el Stack.Screen name="(tabs)"

b) Cambiar el `Stack` raíz por un `Slot`

c) Borrar el `_layout.tsx` de `(tabs)`

d) Navegar con `router.replace` en lugar de `router.push`

9. ¿Qué opción de `Stack.Screen` abre una pantalla como modal?

a) `modal: true`

b) presentation: 'modal'

c) `animation: 'modal'`

d) `type: 'modal'`

10. ¿Cuál de estas afirmaciones es **falsa**?

a) Cada `_layout.tsx` define el navegador de las rutas de su carpeta.

b) Un `Drawer` puede tener un `Tabs` como una de sus pantallas.

c) El nombre de un grupo entre paréntesis aparece en la URL.

d) Cada pestaña de `Tabs` conserva su estado al cambiar de pestaña.

1	2	3	4	5	6	7	8	9	10

A2. Preguntas abiertas (2,5 puntos cada una)

A2.1 Expliquen con sus palabras la diferencia entre `<Slot />` y `<Stack />`. Den un caso real de una app en el que usarían cada uno.

R=/ Slot muestra la pantalla activa dentro de un diseño común. Lo usaría en un área de acceso. Stack guarda el recorrido y permite volver. Lo usaría para pasar de una lista al detalle.

A2.2 Una app bancaria tiene estas pantallas: Inicio, Movimientos, Transferir, Tarjetas, Perfil, Ajustes, Ayuda, Términos legales, Detalle de un movimiento y Confirmar transferencia. Propongan qué va en **Tabs**, qué va en el **Drawer** y qué se abre con **Stack** (o como modal). Justifiquen cada decisión en una línea.
R=/ Tabs: Inicio, Movimientos, Transferir y Tarjetas, porque se usan seguido. Drawer: Perfil, Ajustes, Ayuda y Términos. Stack: Detalle y Confirmar transferencia.

Parte B · Práctica (25 puntos)

Van a construir **Biblioteca**, una app pequeña que usa los cuatro layouts tal como en el árbol de la sección 4. No hace falta backend ni estilos elaborados: se evalúa la navegación.

B0. Preparación (2 puntos)

```
npx create-expo-app@latest taller-nav
cd taller-nav
npm run reset-project          # respondan n: deja src/app/ vacía
npx expo install @react-navigation/drawer @expo/vector-icons
npx expo start
```

> **Borren** `src/app/index.tsx` (lo crea `reset-project`). El inicio será `(app)/(tabs)/index.tsx`; si dejan los dos, ambos compiten por la URL `/`.
>
> La plantilla ya trae `react-native-gesture-handler`, `react-native-reanimated` y `react-native-worklets`. Si trabajan sobre un proyecto anterior, instálenlos también con `npx expo install`.

Creen esta estructura dentro de `src/` (los archivos pueden empezar con un componente que solo devuelva un `<Text>`):

```
src/
├── app/
│   ├── _layout.tsx
│   ├── acerca.tsx
│   ├── libro/
│   │   └── [id].tsx
│   ├── (auth)/
│   │   ├── _layout.tsx
│   │   └── login.tsx
│   └── (app)/
│       ├── _layout.tsx
│       ├── ajustes.tsx
│       └── (tabs)/
│           ├── _layout.tsx
│           ├── index.tsx
│           ├── favoritos.tsx
│           └── perfil.tsx
└── data/
    └── libros.ts            # se importa como '@/data/libros'
```

Copien estos datos en `src/data/libros.ts`:

```
export const LIBROS = [
  { id: '1', titulo: 'Cien años de soledad', autor: 'Gabriel García Márquez', anio: 1967 },
  { id: '2', titulo: 'La vorágine', autor: 'José Eustasio Rivera', anio: 1924 },
  { id: '3', titulo: 'María', autor: 'Jorge Isaacs', anio: 1867 },
  { id: '4', titulo: 'Delirio', autor: 'Laura Restrepo', anio: 2004 },
];
```

B1. Stack raíz (4 puntos)

Completen `app/_layout.tsx`. El Stack raíz debe tener cuatro pantallas:

- `(app)` y `(auth)` sin header (cada una tiene su propio layout).
- `libro/[id]` con el título "Detalle".
- `acerca` presentada como **modal**.

```
import { Stack } from 'expo-router';
import { GestureHandlerRootView } from 'react-native-gesture-handler';

export default function RootLayout() {
  return (
    <GestureHandlerRootView style={{ flex: 1 }}>
      <Stack>
        {/* TODO: cuatro Stack.Screen con sus opciones */}
      </Stack>
    </GestureHandlerRootView>
  );
}
```

B2. Grupo (auth) con Slot (3 puntos)

1. En `app/(auth)/_layout.tsx` usen `<Slot />` dentro de un `View` con un color de fondo y el texto "Biblioteca" arriba. Así cualquier pantalla de autenticación futura comparte ese marco.
2. En `login.tsx` pongan un botón **Entrar** que lleve al inicio (`/`) de forma que **no** se pueda volver al login con el botón atrás.
3. Para probarlo, abran `/login` desde el menú de desarrollo o con un `<Link>` temporal.

B3. Drawer (4 puntos)

En `app/(app)/_layout.tsx` creen un `Drawer` con dos opciones en el menú: **Inicio** (la pantalla `(tabs)`) y **Ajustes**. Pista: `drawerLabel` cambia el texto del menú y `title` el del header.

B4. Tabs (4 puntos)

En `app/(app)/(tabs)/_layout.tsx` creen tres pestañas, **Inicio**, **Favoritos** y **Perfil**, cada una con un ícono de `Ionicons` (de `@expo/vector-icons`, que instalaron en B0). **Debe verse un solo header**, el del Drawer, con el botón de menú.

```
tabBarIcon: ({ color, size }) => <Ionicons name="home" color={color as string} size={size} />
// "as string": en SDK 58 los tipos de color de Tabs e Ionicons no coinciden
```

B5. Lista y detalle con parámetro (5 puntos)

1. En `(tabs)/index.tsx` muestren `LIBROS` con un `FlatList`. Cada fila (título y autor) navega a `/libro/[id]` con `Link` o `router.push`.
2. En `libro/[id].tsx` lean el `id` con `useLocalSearchParams` y muestren título, autor y año. Si el id no existe, muestren "Libro no encontrado".
3. El título del header debe ser el título del libro. Pista: se puede renderizar `<Stack.Screen options={{ ... }} />` dentro del propio componente.
4. Verifiquen que en el detalle **no** se ven ni la barra de pestañas ni el botón del Drawer, y que el botón atrás regresa a la lista.

B6. Modal (2 puntos)

En `perfil.tsx` agreguen un botón **Acerca de** que abra `/acerca`. La pantalla `acerca` debe abrirse como modal y tener un botón **Cerrar** que use `router.back()`.

B7. Cerrar sesión (3 punto)

En `perfil.tsx` agreguen un botón **Cerrar sesión** que lleve a `/login` sin que se pueda volver a la app con el botón atrás.

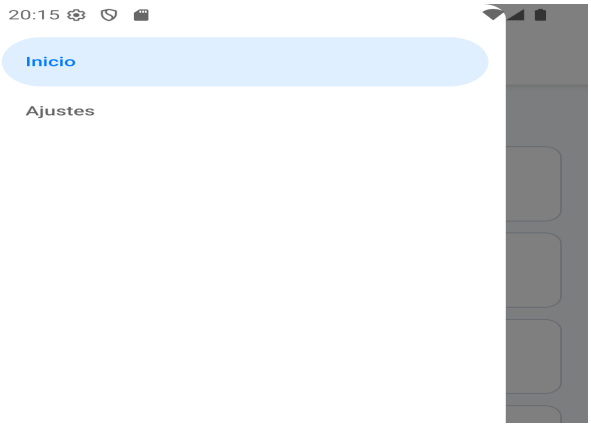
Evidencias

Peguen aquí una captura de pantalla de cada caso:

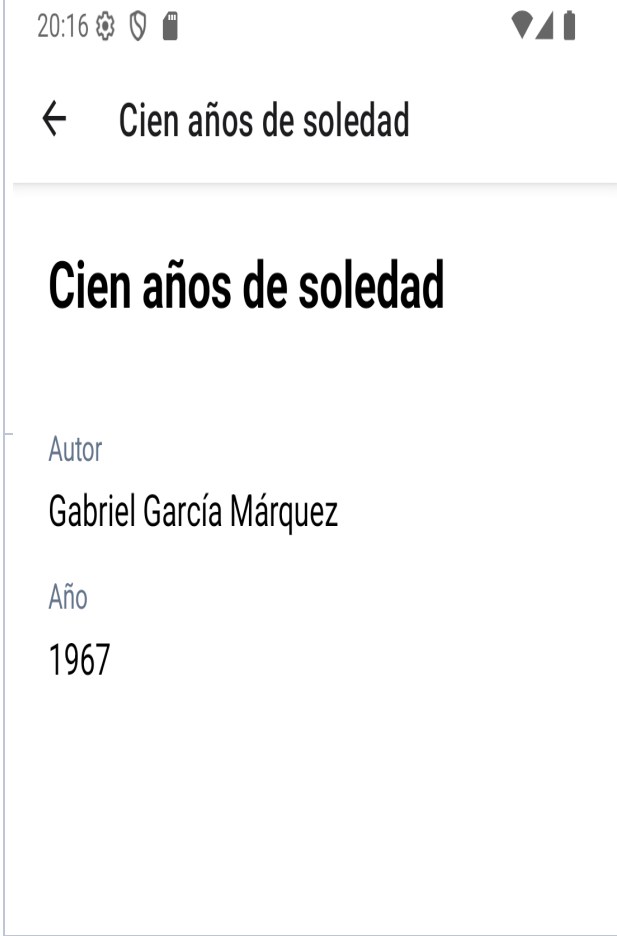
1. Login (Slot con el marco de color)



2. Drawer abierto sobre las pestañas



3. Detalle de un libro con su título en el header



4. Modal “Acerca de”



Enlace al repositorio o nombre del .zip: https://github.com/miguelates279/Biblioteca-recuperacion

Parte C · Depuración (10 min · 10 puntos)

Cada fragmento tiene un error. Expliquen **qué falla** y escriban **la corrección**.

C1 (3 puntos)

```
// app/(auth)/_layout.tsx
import { Slot } from 'expo-router';

export default function AuthLayout() {
  return (
    <Slot>
      <Slot.Screen name="login" options={{ title: 'Ingresar' }} />
    </Slot>
  );
}
```

¿Qué falla y cómo se corrige?

R=/ Slot no tiene Slot.Screen. Se corrige dejando solamente <Slot />.

C2 (3 puntos)

Los archivos de la carpeta son `index.tsx` y `perfil.tsx`. Al abrir la app aparecen **tres** pestañas, una de ellas sin título ni ícono, y en la consola sale una advertencia.

```
// app/(app)/(tabs)/_layout.tsx
import { Tabs } from 'expo-router';

export default function TabsLayout() {
  return (
    <Tabs screenOptions={{ headerShown: false }}>
      <Tabs.Screen name="index" options={{ title: 'Inicio' }} />
      <Tabs.Screen name="Perfil" options={{ title: 'Perfil' }} />
    </Tabs>
  );
}
```

¿Qué falla y cómo se corrige?

R=/ Perfil no coincide con el archivo perfil.tsx. Se corrige usando name="perfil".

C3 (4 puntos)

El detalle está en `app/libro/[id].tsx`. Al tocar un libro aparece la pantalla “Unmatched Route”.

```
// app/(app)/(tabs)/index.tsx
<Pressable onPress={() => router.push('/(tabs)/libro/' + item.id)}>
  <Text>{item.titulo}</Text>
</Pressable>
```

a) ¿Qué falla y cómo se corrige? (3 puntos)

R=/ La dirección apunta a una carpeta equivocada. Se corrige con "/libro/" + item.id.

---

b) Si en cambio movieran el archivo a `app/(app)/(tabs)/libro/[id].tsx`, ¿qué cambiaría en lo que ve el usuario al abrir el detalle? (1 punto)

R=/ El detalle se abriría dentro de las pestañas, así que la barra inferior y el encabezado del menú lateral seguirían visibles.
